

IF2211 - Strategi Algoritma

Tugas Kecil 3

Ice Sliding Puzzle Solver



Faris Wirakusuma Triawan - 13524130

Jonathan Harijadi - 13524144

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung**

2026

Daftar Isi

Bab 1.....	2
Algoritma Pathfinding.....	3
1. Algoritma A*.....	3
1.1. Alur kerja.....	3
1.2. Pseudocode.....	3
2. Algoritma GBFS.....	3
2.1. Alur kerja.....	3
2.2. Pseudocode.....	3
3. Algoritma UCS.....	3
3.1. Alur kerja.....	3
3.2. Pseudocode.....	3
4. Algoritma Dijkstra.....	3
4.1. Alur kerja.....	3
4.2. Pseudocode.....	3
Bab 2.....	4
Analisis Algoritma Pathfinding.....	4
1. Definisi dari $f(n)$ dan $g(n)$ pada algoritma.....	4
2. Apakah heuristik yang digunakan pada algoritma A* admissible?.....	4
3. Secara teoritis, perbandingan algoritma yang diimplementasikan pada permasalahan ini.....	4
Bab 3.....	5
Source Code.....	5
1. File A_Star.cpp.....	5
2. File GBFS.cpp.....	5
3. File Dijkstra.cpp.....	5
4. File UCS.cpp.....	5
Bab 4.....	6
Tangkapan Layar.....	6
Percobaan 1.....	6
Percobaan 2.....	7
Percobaan 3.....	8
Percobaan 4.....	9
Percobaan 5.....	10
Lampiran.....	11

Bab 1

Algoritma Pathfinding

1. Algoritma A*

1.1. Alur kerja

A* adalah algoritma pencarian heuristik (*informed search*) yang mencari rute terpendek dengan mengevaluasi kombinasi antara biaya aktual yang telah dikeluarkan dan estimasi biaya menuju target. Algoritma ini bersifat *complete* (pasti menemukan solusi jika ada) dan *optimal* (pasti menemukan rute termurah) selama fungsi heuristiknya bersifat *admissible* (tidak pernah melebihi-lebihkan jarak asli).

Fungsi Evaluasi:

$$f(n) = g(n) + h(n)$$

$g(n)$: : Biaya aktual dari *start* ke *node n*.

$h(n)$:Estimasi biaya (heuristik) dari *node n* ke *target*.

1.2. Pseudocode

```
function A_Star(start, target):
    openSet = PriorityQueue() # Min-heap berdasarkan nilai f(n)
    openSet.push(start, f=0)
    gScore = Map(default=infinity)
    gScore[start] = 0

    while not openSet.isEmpty():
        current = openSet.pop()

        if current == target:
            return reconstructPath(current)

        for neighbor in getNeighbors(current):
            tentative_g = gScore[current] + cost(current, neighbor)

            if tentative_g < gScore[neighbor]:
                gScore[neighbor] = tentative_g
                h = calculateHeuristic(neighbor, target)
                f = tentative_g + h
                openSet.push(neighbor, f)
                parentMap[neighbor] = current
```

```
return FAILURE
```

2. Algoritma GBFS

2.1. Alur kerja

GBFS adalah algoritma *informed search* yang secara eksklusif berfokus pada pendekatan menuju target secepat mungkin. Algoritma ini mengabaikan biaya jalan yang sudah ditempuh dan murni mengevaluasi *node* berdasarkan nilai heuristiknya. GBFS tidak *complete* dan tidak *optimal* karena rentan terjebak dalam *local optima* (misalnya menabrak jalan buntu/tembok karena rute tersebut terlihat dekat dengan target).

Fungsi Evaluasi:

$$f(n) = h(n)$$

$h(n)$:Estimasi biaya (heuristik) dari *node n* ke *target*.

2.2. Pseudocode

```
function GBFS(start, target):
    openSet = PriorityQueue() # Min-heap berdasarkan nilai h(n)
    openSet.push(start, h=calculateHeuristic(start, target))
    visited = Set()

    while not openSet.isEmpty():
        current = openSet.pop()

        if current == target:
            return reconstructPath(current)

        visited.add(current)

        for neighbor in getNeighbors(current):
            if neighbor not in visited:
                h = calculateHeuristic(neighbor, target)
                openSet.push(neighbor, h)
                parentMap[neighbor] = current
                visited.add(neighbor) # Hindari loop

    return FAILURE
```

3. Algoritma UCS

3.1. Alur kerja

UCS adalah algoritma *uninformed search* (tanpa heuristik) yang mengeksplorasi graf berdasarkan biaya kumulatif paling rendah. Algoritma ini akan menyebar ke segala arah secara radial dari titik awal. UCS bersifat *complete* dan *optimal* pada graf dengan bobot *edge* non-negatif.

Fungsi Evaluasi:

$$f(n) = g(n)$$

$g(n)$: : Biaya aktual dari *start* ke *node n*.

3.2. Pseudocode

```
function UCS(start, target):
    openSet = PriorityQueue() # Min-heap berdasarkan nilai g(n)
    openSet.push(start, g=0)
    gScore = Map(default=infinity)
    gScore[start] = 0
    visited = Set()

    while not openSet.isEmpty():
        current = openSet.pop()

        if current == target:
            return reconstructPath(current)

        if current in visited:
            continue
        visited.add(current)

        for neighbor in getNeighbors(current):
            tentative_g = gScore[current] + cost(current, neighbor)

            if tentative_g < gScore[neighbor]:
                gScore[neighbor] = tentative_g
                openSet.push(neighbor, tentative_g)
                parentMap[neighbor] = current

    return FAILURE
```

4. Algoritma Dijkstra

4.1. Alur kerja

Dalam konteks pencarian jalur dari satu titik ke titik target (*Single-Source Shortest Path*), mekanisme Dijkstra beroperasi secara identik dengan UCS. Algoritma ini menggunakan struktur *priority queue* untuk merelaksasi *edge* dan mencari jalur dengan akumulasi biaya terkecil tanpa panduan heuristik. Perbedaannya biasanya hanya pada literatur: Dijkstra sering ditulis untuk memetakan jarak ke *seluruh node* dalam graf, sedangkan UCS menghentikan pencarian tepat saat *target* ditemukan.

Fungsi Evaluasi:

$$f(n) = g(n)$$

$g(n)$: : Biaya aktual dari *start* ke *node n*.

4.2. Pseudocode

```
function Dijkstra(graph, start, target):
    dist = Map(default=infinity)
    dist[start] = 0
    openSet = PriorityQueue() # Min-heap berdasarkan jarak
    openSet.push(start, dist[start])

    while not openSet.isEmpty():
        current = openSet.pop()

        # Kondisi terminasi awal untuk pencarian point-to-point (opsional)
        if current == target:
            return reconstructPath(current)

        for neighbor in graph.getNeighbors(current):
            alt_path = dist[current] + cost(current, neighbor)

            # Proses Relaksasi (Relaxation)
            if alt_path < dist[neighbor]:
                dist[neighbor] = alt_path
                parentMap[neighbor] = current
                openSet.push(neighbor, alt_path)

    return FAILURE
```

Bab 2

Analisis Algoritma Pathfinding

1. Definisi dari $f(n)$ dan $g(n)$ pada algoritma pathfinding

Dalam Ice Sliding Puzzle Solver ini, definisi kedua function tersebut adalah sebagai berikut:

- $g(n)$: function ini merupakan *actual cost*/biaya kumulatif nyata yang dikeluarkan dari titik start menuju node n . $g(n)$ dihitung berdasarkan jumlah total cost seluruh tile yang dilalui selama satu gerakan meluncur. Jika aktor melewati 5 tile sebelum menabrak dinding, maka $g(n)$ adalah penjumlahan biaya dari kelima tile tersebut (bukan hanya tile tempat aktor berhenti).
- $h(n)$: function ini merupakan *heuristic cost*/estimasi biaya atau jarak dari node n menuju target terdekat.
- $f(n)$: function ini merupakan *total estimated cost*/fungsi evaluasi total yang menentukan urutan prioritas node. Function $f(n)$ berbeda sesuai dengan algoritma pathfinding yang digunakan. Pada algoritma Dijkstra dan UCS, $f(n) = g(n)$ sehingga algoritma hanya melihat cost aslinya saja. Sebaliknya pada algoritma GBFS, $f(n) = h(n)$ sehingga algoritma hanya melihat estimasi jarak atau heuristic costnya. Terakhir pada algoritma A^* , $f(n) = g(n) + h(n)$ sehingga algoritma ini menggunakan gabungan biaya nyata dengan estimasi jarak.

2. Apakah heuristik yang digunakan pada algoritma A^* admissible?

Heuristic yang digunakan algoritma A^* akan dikatakan admissible jika algoritma ini bisa menghasilkan rute atau hasil yang optimal dan tidak pernah memberikan estimasi yang lebih besar daripada biaya sebenarnya. Pada kasus Ice Sliding Puzzle Solver ini, biaya/cost dapat bernilai 0 sehingga penggunaan heuristik jarak geometris seperti Manhattan dan Euclidean memiliki kemungkinan menghasilkan nilai $h(n)$ atau estimasi jarak lebih besar daripada nilai $g(n)$ atau cost aslinya sehingga menyebabkan algoritma A^* menjadi non-admissible.

Algoritma A^* hanya bisa dipastikan admissible jika minimal cost pada setiap tile adalah 1 dikarenakan jarak lurus yang dihasilkan heuristik akan selalu lebih kecil daripada biaya asli yang berliku-liku. Tetapi karena minimal costnya 0, maka nilai estimasi jarak yang dihasilkan bisa melewati biaya aslinya jika contoh melewati sekumpulan tile bernilai 0 yang sudah dipastikan nilai estimasi $h(n) = n$ akan lebih besar dari biaya asli $g(n) = 0$ sehingga algoritma A^* non-admissible.

3. Secara teoritis, perbandingan algoritma yang diimplementasikan pada permasalahan ini.

Kriteria	Algoritma A^*	Algoritma GBFS	Algoritma UCS	Algoritma Dijkstra
----------	-----------------	----------------	---------------	--------------------

Fungsi Evaluasi	$f(n) = g(n) + h(n)$	$f(n) = h(n)$	$f(n) = g(n)$	$f(n) = g(n)$
Optimalitas	Optimal Terkondisi. Optimal jika heuristik admissible. Heuristik dapat dipastikan admissible jika minimal cost tile lebih besar dari 0	Tidak Optimal. Seringkali menemukan jalur yang mahal.	Optimal. Menjamin jalur dengan total biaya terendah.	Optimal. Menjamin ditemukan jalur dengan total cost paling rendah.
Efisiensi Waktu	Moderat - Tinggi. Lebih efisien dari UCS karena dipandu heuristik.	Sangat Tinggi. Langsung menuju target, mengabaikan biaya.	Rendah. Mengeksplorasi node secara menyeluruh ke segala arah.	Rendah. Mengeksplorasi node secara melingkar, sehingga banyak node yang tidak relevan ikut diperiksa.
Karakteristik & Kesesuaian pada Peta Ice Puzzle	Pilihan terbaik untuk Map Kompleks. Sangat bagus untuk peta yang memerlukan keseimbangan antara skor (cost rendah) dan efisiensi waktu pencarian (jumlah iterasi sedikit).	Sangat bagus untuk Map yang sangat luas dengan biaya tile seragam. Jika semua kotak biayanya sama, GBFS akan menemukan jalan keluar jauh lebih cepat daripada algoritma lain.	Sangat bagus untuk Map dengan variasi biaya ekstrim. Jika peta memiliki banyak jalur "gratis" (cost 0) yang berliku dan jalur langsung yang sangat mahal, UCS pasti menemukan jalur termurah.	Cocok untuk Map tanpa Heuristik. Digunakan jika kita tidak bisa menentukan jarak estimasi ke target (misal target berpindah-pindah secara acak).

Bab 3

Source Code

1. File A_Star.cpp

```
#include <A_Star.hpp>

double A_Star_Solver::calculate_h(cord current, int currentSeq) {
    cord currentTarget;
    auto it = p_mapData->waypoints.find(currentSeq);

    if (it != p_mapData->waypoints.end()) {
        currentTarget = it->second;
    } else {
        currentTarget = p_mapData->target;
    }

    double dx = (double)abs(current.first - currentTarget.first);
    double dy = (double)abs(current.second - currentTarget.second);

    switch (heuristic) {
        case HEUR_MANHATTAN:
            return dx + dy;

        case HEUR_EUCLIDEAN:
            return sqrt(dx * dx + dy * dy);

        case HEUR_CHEBYSHEV:
            return (dx > dy) ? dx : dy;

        default:
            return 0;
    }
}

double A_Star_Solver::calculate_g(double parent_g, cord current, cord neighbor) {
    double stepCost = 0;
    int dr = (neighbor.first > current.first) - (neighbor.first < current.first);
    int dc = (neighbor.second > current.second) - (neighbor.second < current.second);

    int r = current.first;
    int c = current.second;

    while (r != neighbor.first || c != neighbor.second) {
        r += dr;
        c += dc;
        stepCost += p_mapData->costMap[r][c];
    }
}
```

```

    return parent_g + stepCost;
}

void A_Star_Solver::expandNode(Node current) {
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    for (int i = 0; i < 4; i++) {
        int r = current.c.first;
        int c = current.c.second;
        int seq = current.seq;
        bool validStop = false;

        while (true) {
            int nr = r + dr[i];
            int nc = c + dc[i];
            if (nr < 0 || nr >= p_mapData->rows || nc < 0 || nc >= p_mapData->cols) {
                validStop = true;
                break;
            }

            if (p_mapData->grid[nr][nc] == 'X') {
                validStop = true;
                break;
            }

            r = nr; c = nc;
            char cell = p_mapData->grid[r][c];

            if (cell == 'L') {
                validStop = false;
                break;
            }

            if (isdigit(cell)) {
                int val = cell - '0';
                if (val == seq) seq++;
            }
        }

        if (validStop && (r != current.c.first || c != current.c.second)) {
            cord nextPos = {r, c};
            State nextState = {r, c, seq};

            currentExploringEdges.push_back({current.c, nextPos});

            double new_g = calculate_g(current.g, current.c, nextPos);
            if (gScore.find(nextState) == gScore.end() || new_g < gScore[nextState]) {
                gScore[nextState] = new_g;
            }
        }
    }
}

```

```

        parentMap[nextState] = {current.c.first, current.c.second, current.seq};

        double h_val = calculate_h(nextPos, seq);
        openSet.push(Node(nextPos, seq, new_g, h_val, new_g + h_val));
    }
}
}
}

bool A_Star_Solver::isFinished() {
    return found || openSet.empty();
}

vector<cord> A_Star_Solver::getOpenSetCoords() const {
    vector<cord> coords;
    const auto& container = openSet.get_container();
    for (const auto& node : container) {
        coords.push_back(node.c);
    }
    return coords;
}

vector<cord> A_Star_Solver::getClosedSetCoords() const {
    set<cord> uniqueCoords;

    for (const auto& s : closedSet) {
        uniqueCoords.insert({s.r, s.c});
    }

    return vector<cord>(uniqueCoords.begin(), uniqueCoords.end());
}

vector<cord> A_Star_Solver::getFinalPathCoords() const {
    vector<cord> path;
    if (!found) return path;

    State current = this->lastState;
    int safety = 0;
    int maxLimit = p_mapData->rows * p_mapData->cols * ((int)p_mapData->waypoints.size() + 1);

    while (safety < maxLimit) {
        path.push_back({current.r, current.c});

        if (current.r == start.first && current.c == start.second && current.seq == 0) break;

        auto it = parentMap.find(current);
        if (it == parentMap.end()) break;

        current = it->second;
        safety++;
    }
}

```

```

    }

    std::reverse(path.begin(), path.end());
    return path;
}

void A_Star_Solver::step() {
    currentExploringEdges.clear();
    if (!openSet.empty() && !found) {
        Node current = openSet.top();
        openSet.pop();

        State currentState = {current.c.first, current.c.second, current.seq};

        auto itBest = gScore.find(currentState);
        if (itBest != gScore.end() && current.g > itBest->second) {
            return;
        }
        if (closedSet.find(currentState) != closedSet.end()) return;

        if (current.c.first == p_mapData->target.first &&
            current.c.second == p_mapData->target.second &&
            current.seq == (int)p_mapData->waypoints.size()) {

            this->lastState = currentState;
            this->found = true;
            this->totalCost = current.g;
            return;
        }

        closedSet.insert(currentState);
        expandNode(current);
    }
}

A_Star_Solver::A_Star_Solver(MapData* m, HeuristicType heuType)
:PathFinder(m), heuristic(heuType){
    this->start = m->start;
    this->end = m->target;
    this->found = false;

    int initialSeq = 0;
    double h_val = calculate_h(start, initialSeq);

    State startState = {start.first, start.second, initialSeq};
    gScore[startState] = 0.0;

    openSet.push(Node(start, initialSeq, 0.0, h_val, h_val));
}

```

2. File GBFS.cpp

```
#include <GBFS.hpp>

GBFS_Solver::GBFS_Solver(MapData* m, HeuristicType hType) : Pathfinder(m), heuristic(hType) {
    double h_val = calculate_h(start, 0);
    openSet.push(Node(start, 0, 0.0, h_val, h_val));
    actualG[{start.first, start.second, 0}] = 0.0;
}

double GBFS_Solver::calculate_h(cord current, int currentSeq) {
    cord target = (p_mapData->waypoints.count(currentSeq)) ? p_mapData->waypoints.at(currentSeq) :
p_mapData->target;
    double dx = abs(current.first - target.first);
    double dy = abs(current.second - target.second);
    if (heuristic == HEUR_MANHATTAN) return dx + dy;
    if (heuristic == HEUR_EUCLIDEAN) return sqrt(dx*dx + dy*dy);
    return max(dx, dy);
}

void GBFS_Solver::expandNode(Node current) {
    int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
    for (int i = 0; i < 4; i++) {
        int r = current.c.first, c = current.c.second, seq = current.seq;
        double moveCost = 0;
        bool valid = false;
        while (true) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr < 0 || nr >= p_mapData->rows || nc < 0 || nc >= p_mapData->cols ||
p_mapData->grid[nr][nc] == 'X') {
                valid = true; break;
            }
            r = nr; c = nc;
            if (p_mapData->grid[r][c] == 'L') { valid = false; break; }
            moveCost += p_mapData->costMap[r][c];
            if (isdigit(p_mapData->grid[r][c]) && (p_mapData->grid[r][c] - '0' == seq)) seq++;
        }
        if (valid && (r != current.c.first || c != current.c.second)) {
            State nextS = {r, c, seq};
            if (!closedSet.count(nextS)) {
                double h_val = calculate_h({r, c}, seq);
                parentMap[nextS] = {current.c.first, current.c.second, current.seq};
                actualG[nextS] = actualG[{current.c.first, current.c.second, current.seq}] + moveCost;
                openSet.push(Node({r, c}, seq, actualG[nextS], h_val, h_val));
                currentExploringEdges.push_back({current.c, {r, c}});
            }
        }
    }
}
```

```

    }
}

void GBFS_Solver::step() {
    currentExploringEdges.clear();
    if (openSet.empty() || found) return;
    Node current = openSet.top(); openSet.pop();
    State currS = {current.c.first, current.c.second, current.seq};
    if (closedSet.count(currS)) return;
    if (current.c == p_mapData->target && current.seq == (int)p_mapData->waypoints.size()) {
        found = true; lastState = currS; totalCost = actualG[currS]; return;
    }
    closedSet.insert(currS);
    expandNode(current);
}

bool GBFS_Solver::isFinished() { return found || openSet.empty(); }
vector<cord> GBFS_Solver::getOpenSetCoords() const {
    vector<cord> res;
    for(auto const& n : openSet.get_container()) res.push_back(n.c);
    return res;
}
vector<cord> GBFS_Solver::getClosedSetCoords() const {
    set<cord> u; for(auto const& s : closedSet) u.insert({s.r, s.c});
    return vector<cord>(u.begin(), u.end());
}
vector<cord> GBFS_Solver::getFinalPathCoords() const {
    vector<cord> p; if(!found) return p;
    State c = lastState;
    while(!(c.r == start.first && c.c == start.second && c.seq == 0)) {
        p.push_back({c.r, c.c}); c = parentMap.at(c);
    }
    p.push_back(start); reverse(p.begin(), p.end()); return p;
}
}

```

3. File UCS.cpp

```

#include <UCS.hpp>

#include <algorithm>
#include <cctype>

double UCS_Solver::calculate_g(double parent_g, cord current, cord neighbor) {
    double stepCost = 0.0;
    int dr = (neighbor.first > current.first) - (neighbor.first < current.first);
    int dc = (neighbor.second > current.second) - (neighbor.second < current.second);

```

```

int r = current.first;
int c = current.second;

while (r != neighbor.first || c != neighbor.second) {
    r += dr;
    c += dc;
    stepCost += p_mapData->costMap[r][c];
}

return parent_g + stepCost;
}

void UCS_Solver::expandNode(Node current) {
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    for (int i = 0; i < 4; i++) {
        int r = current.c.first;
        int c = current.c.second;
        int seq = current.seq;
        bool validStop = false;

        while (true) {
            int nr = r + dr[i];
            int nc = c + dc[i];

            if (nr < 0 || nr >= p_mapData->rows || nc < 0 || nc >= p_mapData->cols) {
                validStop = true;
                break;
            }

            if (p_mapData->grid[nr][nc] == 'X') {
                validStop = true;
                break;
            }

            r = nr; c = nc;
            char cell = p_mapData->grid[r][c];

            if (cell == 'L') {
                validStop = false;
                break;
            }
            if (isdigit(cell)) {
                int val = cell - '0';
                if (val == seq) seq++;
            }
        }

        if (validStop && (r != current.c.first || c != current.c.second)) {

```

```

        cord nextPos = {r, c};
        State nextState = {r, c, seq};

        currentExploringEdges.push_back({current.c, nextPos});

        double new_g = calculate_g(current.g, current.c, nextPos);

        if (gScore.find(nextState) == gScore.end() || new_g < gScore[nextState]) {
            gScore[nextState] = new_g;
            parentMap[nextState] = {current.c.first, current.c.second, current.seq};

            openSet.push(Node(nextPos, seq, new_g, 0.0, new_g));
        }
    }
}

void UCS_Solver::step() {
    currentExploringEdges.clear();

    if (!openSet.empty() && !found) {
        Node current = openSet.top();
        openSet.pop();

        State currentState = {current.c.first, current.c.second, current.seq};
        auto itBest = gScore.find(currentState);
        if (itBest != gScore.end() && current.g > itBest->second) {
            return;
        }

        if (closedSet.find(currentState) != closedSet.end()) return;

        if (current.c.first == p_mapData->target.first &&
            current.c.second == p_mapData->target.second &&
            current.seq == (int)p_mapData->waypoints.size()) {

            this->lastState = currentState;
            this->found = true;
            this->totalCost = current.g;
            return;
        }

        closedSet.insert(currentState);
        expandNode(current);
    }
}

bool UCS_Solver::isFinished() {
    return found || openSet.empty();
}

```



```

std::vector<cord> UCS_Solver::getOpenSetCoords() const {
    std::vector<cord> coords;
    const auto& container = openSet.get_container();
    for (const auto& node : container) {
        coords.push_back(node.c);
    }
    return coords;
}

std::vector<cord> UCS_Solver::getClosedSetCoords() const {
    std::set<cord> uniqueCoords;
    for (const auto& s : closedSet) {
        uniqueCoords.insert({s.r, s.c});
    }
    return std::vector<cord>(uniqueCoords.begin(), uniqueCoords.end());
}

std::vector<cord> UCS_Solver::getFinalPathCoords() const {
    std::vector<cord> path;
    if (!found) return path;

    State current = this->lastState;
    int safety = 0;
    int maxLimit = p_mapData->rows * p_mapData->cols * ((int)p_mapData->waypoints.size() + 1);

    while (safety < maxLimit) {
        path.push_back({current.r, current.c});

        if (current.r == start.first && current.c == start.second && current.seq == 0) break;

        auto it = parentMap.find(current);
        if (it == parentMap.end()) break;

        current = it->second;
        safety++;
    }

    std::reverse(path.begin(), path.end());
    return path;
}

UCS_Solver::UCS_Solver(MapData* m) : Pathfinder(m) {
    int initialSeq = 0;
    State startState = {start.first, start.second, initialSeq};
    gScore[startState] = 0.0;
    openSet.push(Node(start, initialSeq, 0.0, 0.0, 0.0));
}

```

4. File Djikstra.cpp

```
#include <Djikstra.hpp>

Djikstra_Solver::Djikstra_Solver(MapData* m) : PathFinder(m) {
    this->found = false;
    State startState = {start.first, start.second, 0};
    gScore[startState] = 0.0;
    openSet.push(Node(start, 0, 0.0, 0.0, 0.0));
}

double Djikstra_Solver::calculate_g(double parent_g, cord current, cord neighbor) {
    double stepCost = 0;
    int dr = (neighbor.first > current.first) - (neighbor.first < current.first);
    int dc = (neighbor.second > current.second) - (neighbor.second < current.second);
    int r = current.first;
    int c = current.second;

    while (r != neighbor.first || c != neighbor.second) {
        r += dr; c += dc;
        stepCost += p_mapData->costMap[r][c];
    }
    return parent_g + stepCost;
}

void Djikstra_Solver::expandNode(Node current) {
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    for (int i = 0; i < 4; i++) {
        int r = current.c.first, c = current.c.second, seq = current.seq;
        bool validStop = false;

        while (true) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr < 0 || nr >= p_mapData->rows || nc < 0 || nc >= p_mapData->cols) {
                validStop = true; break;
            }
            if (p_mapData->grid[nr][nc] == 'X') {
                validStop = true; break;
            }
            r = nr; c = nc;
            if (p_mapData->grid[r][c] == 'L') { validStop = false; break; }
            if (isdigit(p_mapData->grid[r][c])) {
                if ((p_mapData->grid[r][c] - '0') == seq) seq++;
            }
        }
    }
}
```

```

        if (validStop && (r != current.c.first || c != current.c.second)) {
            cord nextPos = {r, c};
            State nextState = {r, c, seq};
            double new_g = calculate_g(current.g, current.c, nextPos);

            if (gScore.find(nextState) == gScore.end() || new_g < gScore[nextState]) {
                gScore[nextState] = new_g;
                parentMap[nextState] = {current.c.first, current.c.second, current.seq};
                openSet.push(Node(nextPos, seq, new_g, 0.0, new_g));
                currentExploringEdges.push_back({current.c, nextPos});
            }
        }
    }
}

void Djikstra_Solver::step() {
    currentExploringEdges.clear();
    if (openSet.empty() || found) return;

    Node current = openSet.top();
    openSet.pop();

    State currentState = {current.c.first, current.c.second, current.seq};
    if (closedSet.count(currentState)) return;
    if (gScore.count(currentState) && current.g > gScore[currentState]) return;

    if (current.c == p_mapData->target && current.seq == (int)p_mapData->waypoints.size()) {
        this->lastState = currentState;
        this->found = true;
        this->totalCost = current.g;
        return;
    }

    closedSet.insert(currentState);
    expandNode(current);
}

bool Djikstra_Solver::isFinished() { return found || openSet.empty(); }

vector<cord> Djikstra_Solver::getOpenSetCoords() const {
    vector<cord> coords;
    for (const auto& node : openSet.get_container()) coords.push_back(node.c);
    return coords;
}

vector<cord> Djikstra_Solver::getClosedSetCoords() const {
    set<cord> unique;
    for (auto const& s : closedSet) unique.insert({s.r, s.c});
    return vector<cord>(unique.begin(), unique.end());
}

```

```
}  
  
vector<cord> Djikstra_Solver::getFinalPathCoords() const {  
    vector<cord> path;  
    if (!found) return path;  
    State curr = lastState;  
    while (!(curr.r == start.first && curr.c == start.second && curr.seq == 0)) {  
        path.push_back({curr.r, curr.c});  
        curr = parentMap.at(curr);  
    }  
    path.push_back(start);  
    reverse(path.begin(), path.end());  
    return path;  
}
```

Bab 4

Tangkapan Layar

Percobaan 1

Input:

7 7

XXXXXXX

X0***X

X**X**X

X***OX

X***1LX

XZ**X*X

XXXXXXX

999 999 999 999 999 999 999

999 3 5 2 8 1 999

999 7 4 999 6 9 999

999 2 8 3 5 4 999

999 6 1 7 2 999 999

999 9 3 4 999 8 999

999 999 999 999 999 999 999

Output:

Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG

Algo : A* Search

Heur : Manhattan (H1)

ENGINE STATS

Iterasi : 13

Waktu : 0.134484 ms

Cost : 0

STATUS: NO SOLUTION

ICE SLIDING SOLVER

CONFIG

Algo : A* Search

Heur : Euclidean (H2)

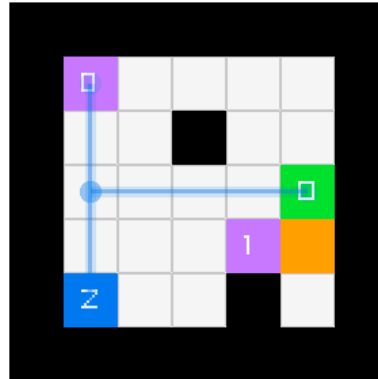
ENGINE STATS

Iterasi : 13

Waktu : 0.151855 ms

Cost : 0

STATUS: NO SOLUTION



ICE SLIDING SOLVER

CONFIG

Algo : A* Search

Heur : Chebyshev (H3)

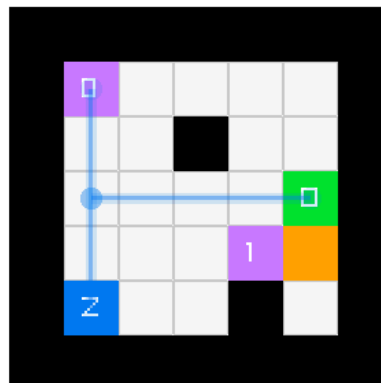
ENGINE STATS

Iterasi : 13

Waktu : 0.125182 ms

Cost : 0

STATUS: NO SOLUTION



ICE SLIDING SOLVER

CONFIG

Algo : BGFS Search

Heur : Manhattan (H1)

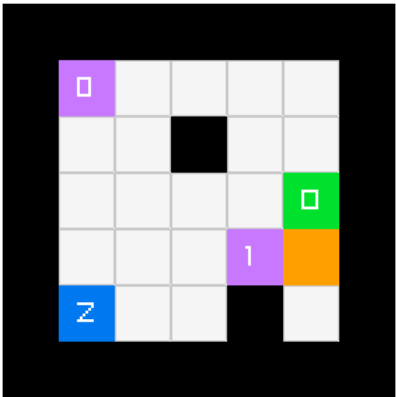
ENGINE STATS

Iterasi : 16

Waktu : 0.158016 ms

Cost : 0

STATUS: NO SOLUTION



ICE SLIDING SOLVER

CONFIG

Algo : BGFS Search

Heur : Euclidean (H2)

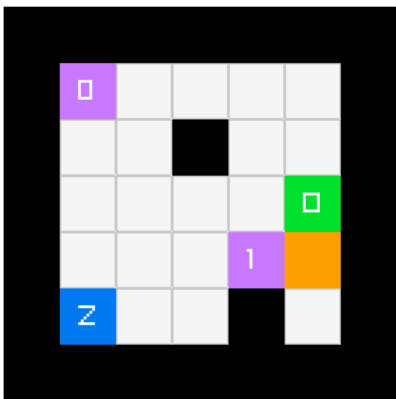
ENGINE STATS

Iterasi : 16

Waktu : 0.143942 ms

Cost : 0

STATUS: NO SOLUTION



Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG

Algo : BGFS Search

Heur : Chebyshev (H3)

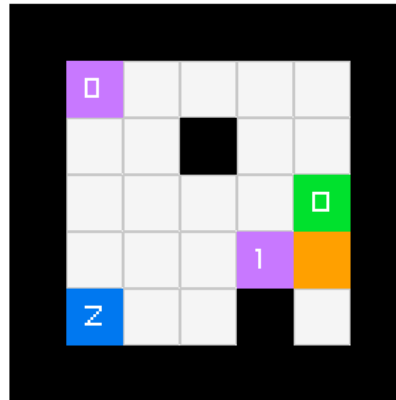
ENGINE STATS

Iterasi : 16

Waktu : 0.152131 ms

Cost : 0

STATUS: NO SOLUTION



Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG

Algo : UCS Search

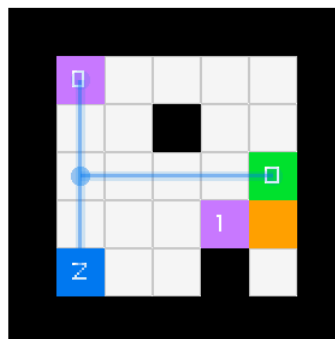
ENGINE STATS

Iterasi : 13

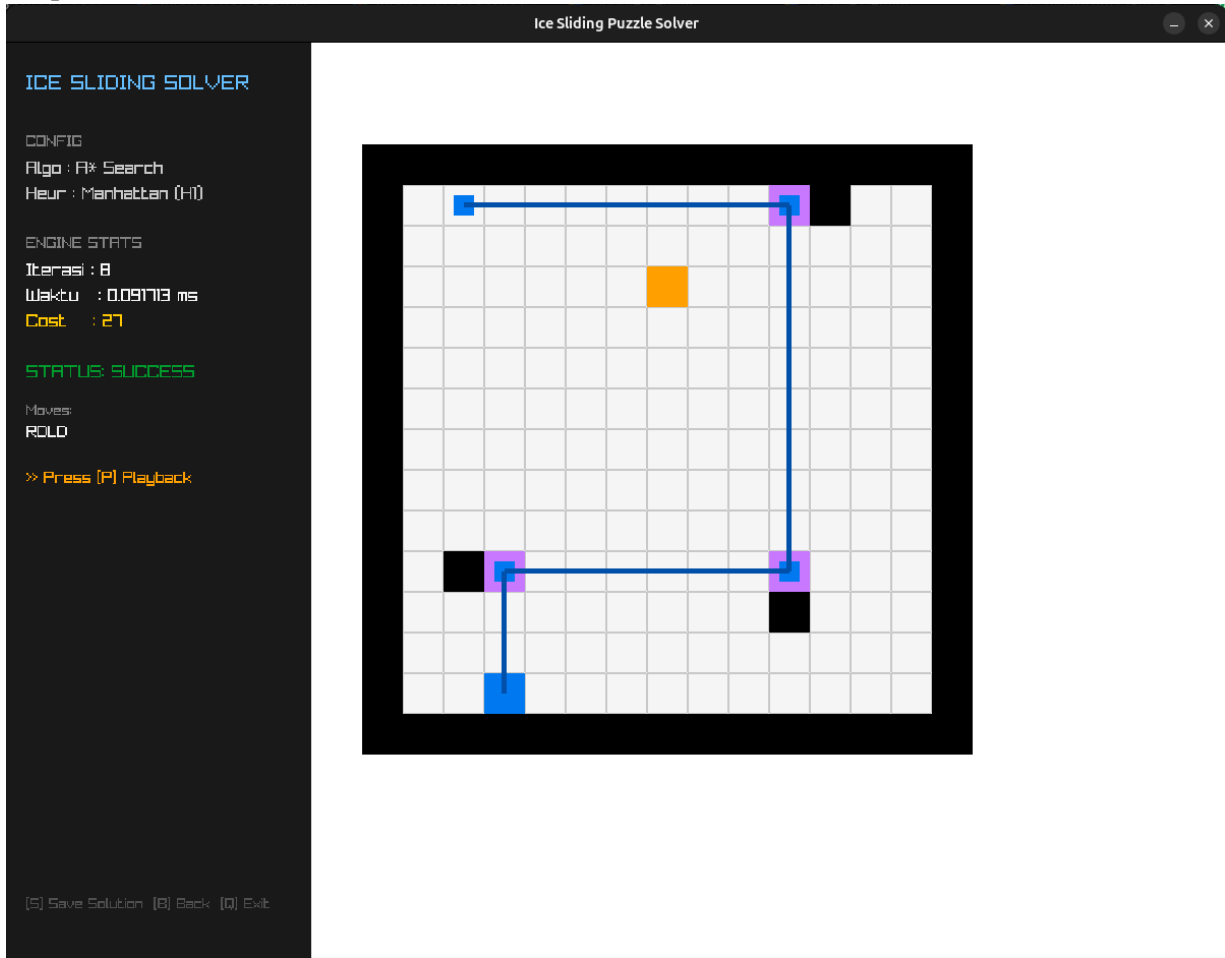
Waktu : 0.114559 ms

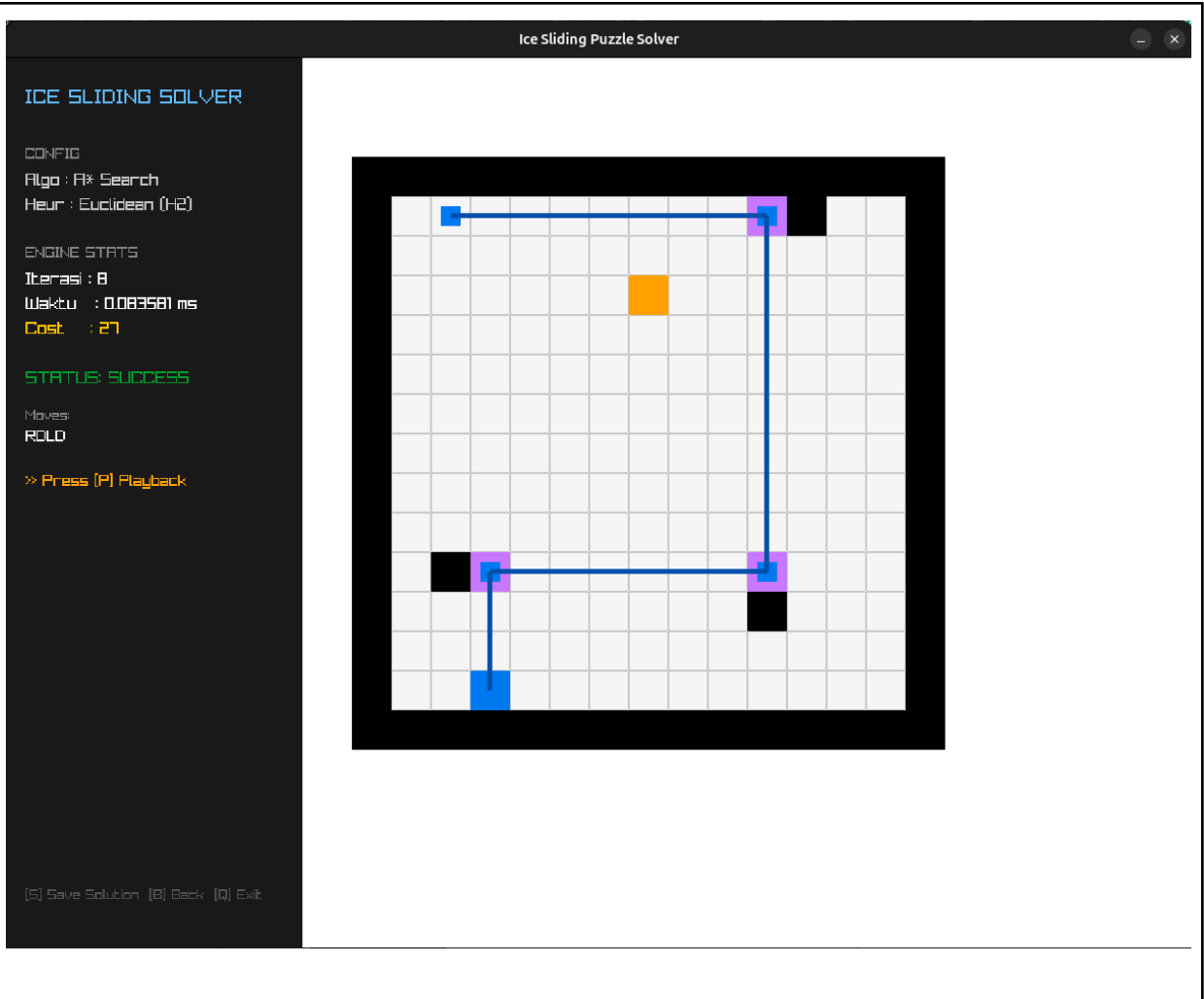
Cost : 0

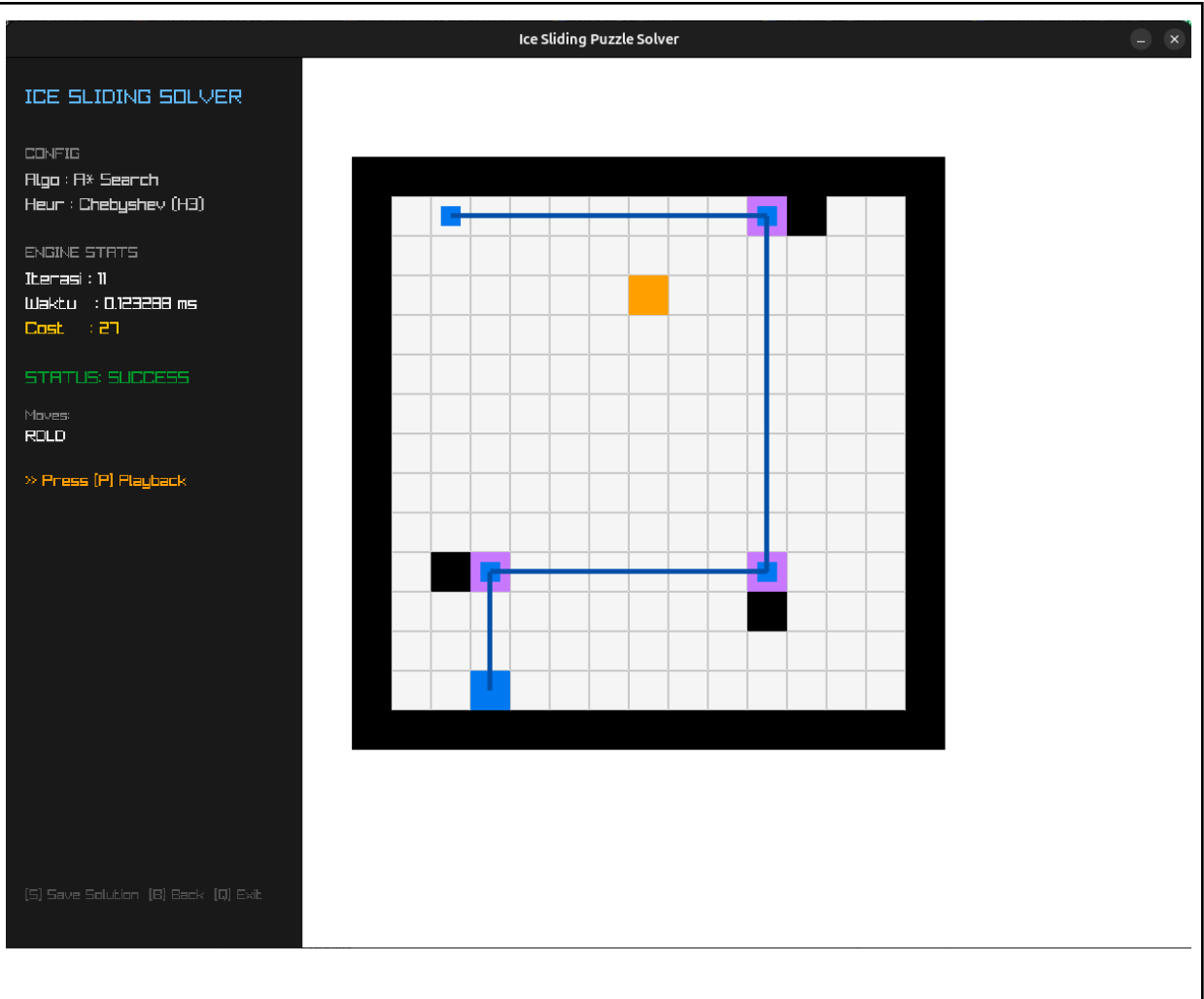
STATUS: NO SOLUTION



Output:







Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG
Algo : BFS Search
Heur : Manhattan (H1)

ENGINE STATS
Iterasi : 6
Waktu : 0.084024 ms
Cost : 29

STATUS: SUCCESS

Moves:
LRDL

>> Press [P] Playback

(S) Save Solution (B) Back (Q) Exit

ICE SLIDING SOLVER

CONFIG

Algo : BGFS Search

Heur : Euclidean (H2)

ENGINE STATS

Iterasi : 6

Waktu : 0.072845 ms

Cost : 29

STATUS: SUCCESS

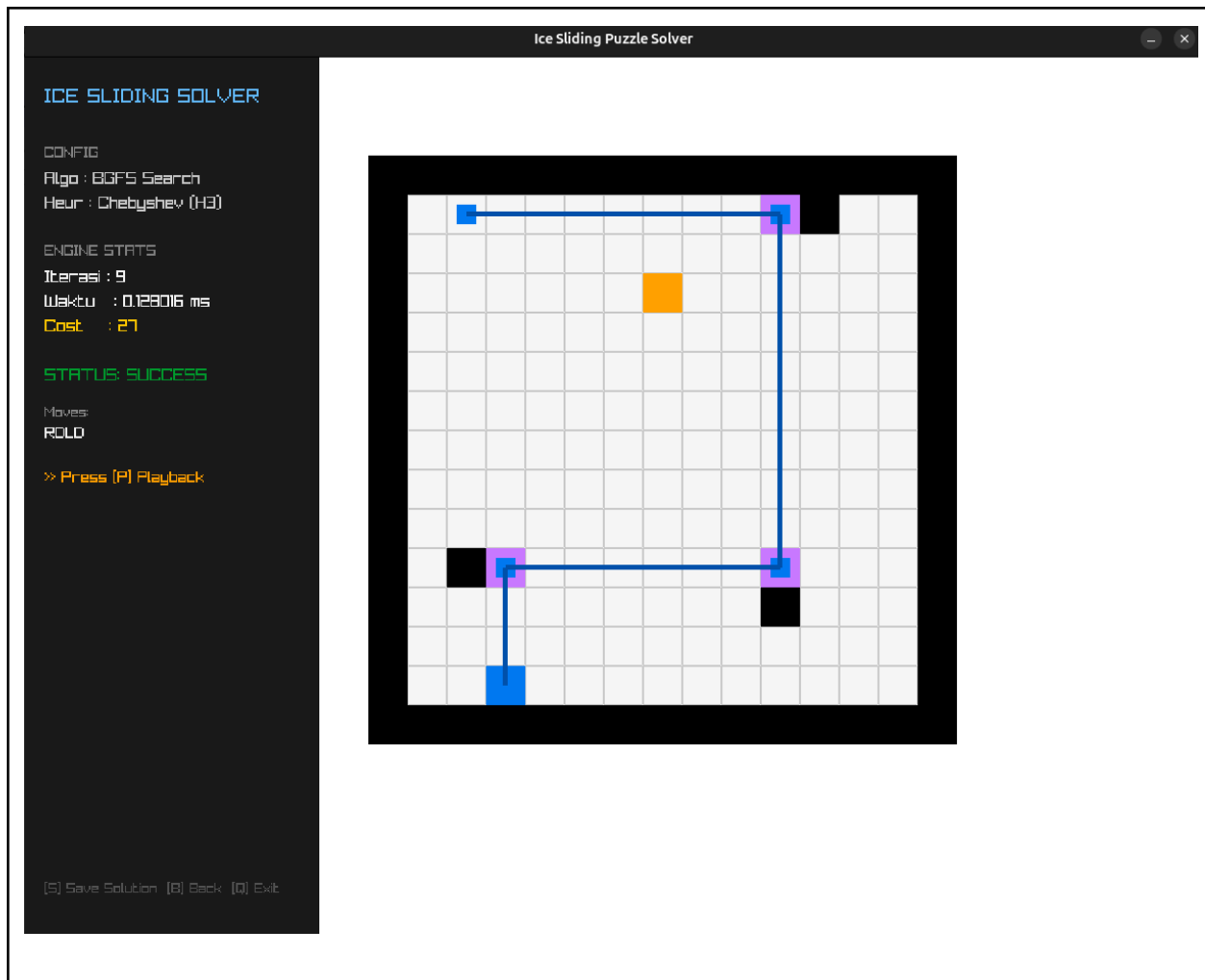
Moves:

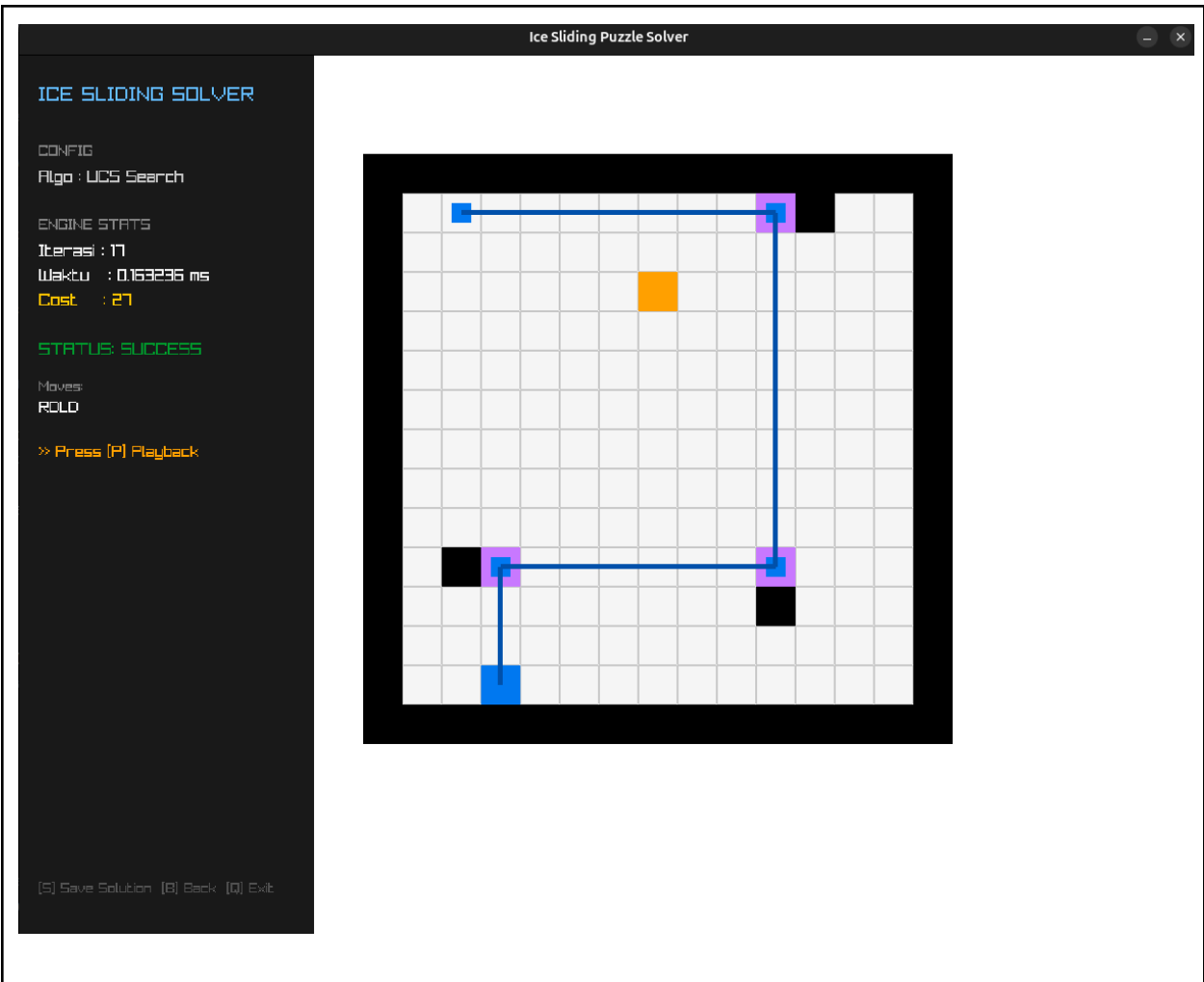
LROLD

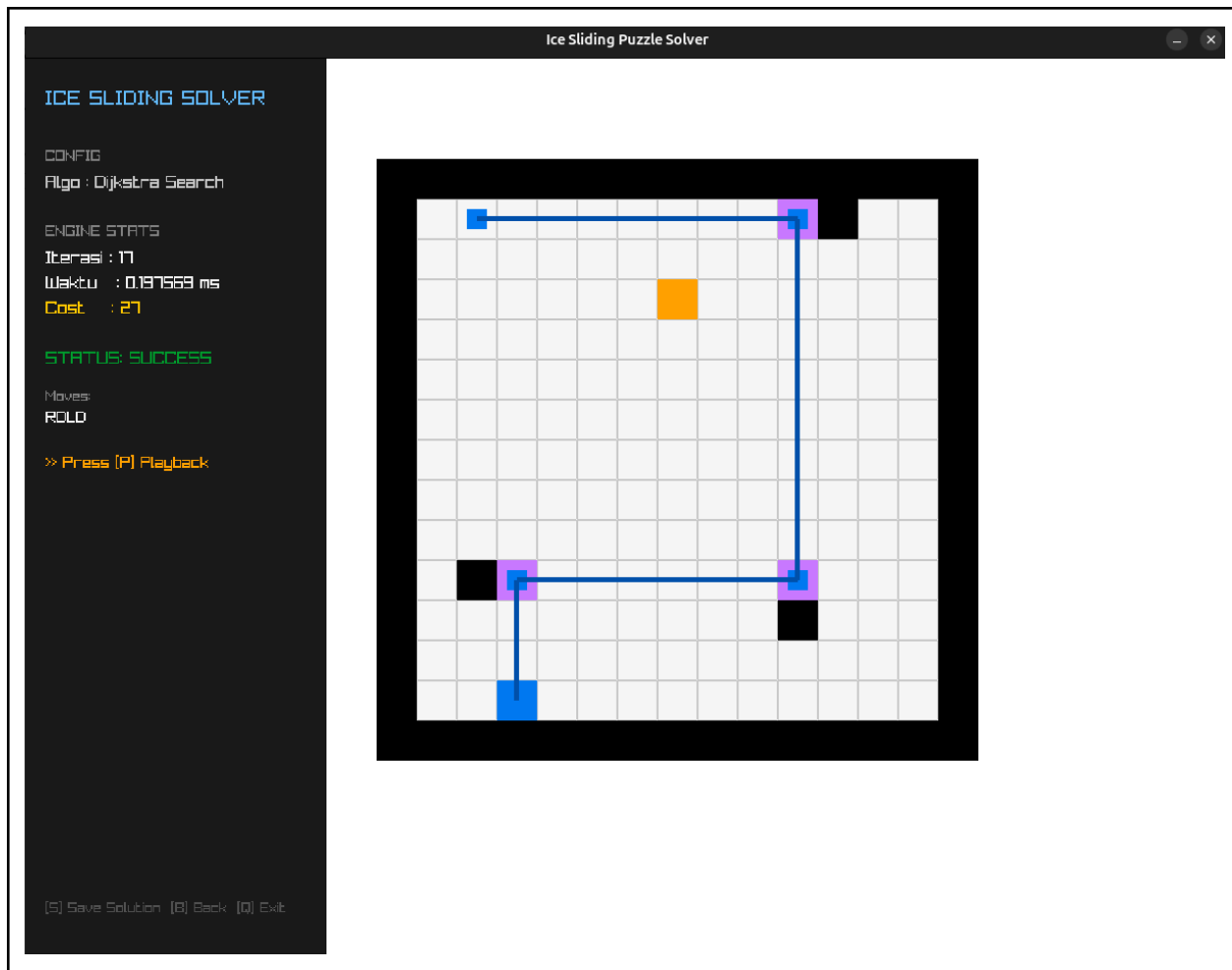
>> Press [P] Playback

[S] Save Solution [B] Back [Q] Exit

Ice Sliding Puzzle Solver







Percobaan 3

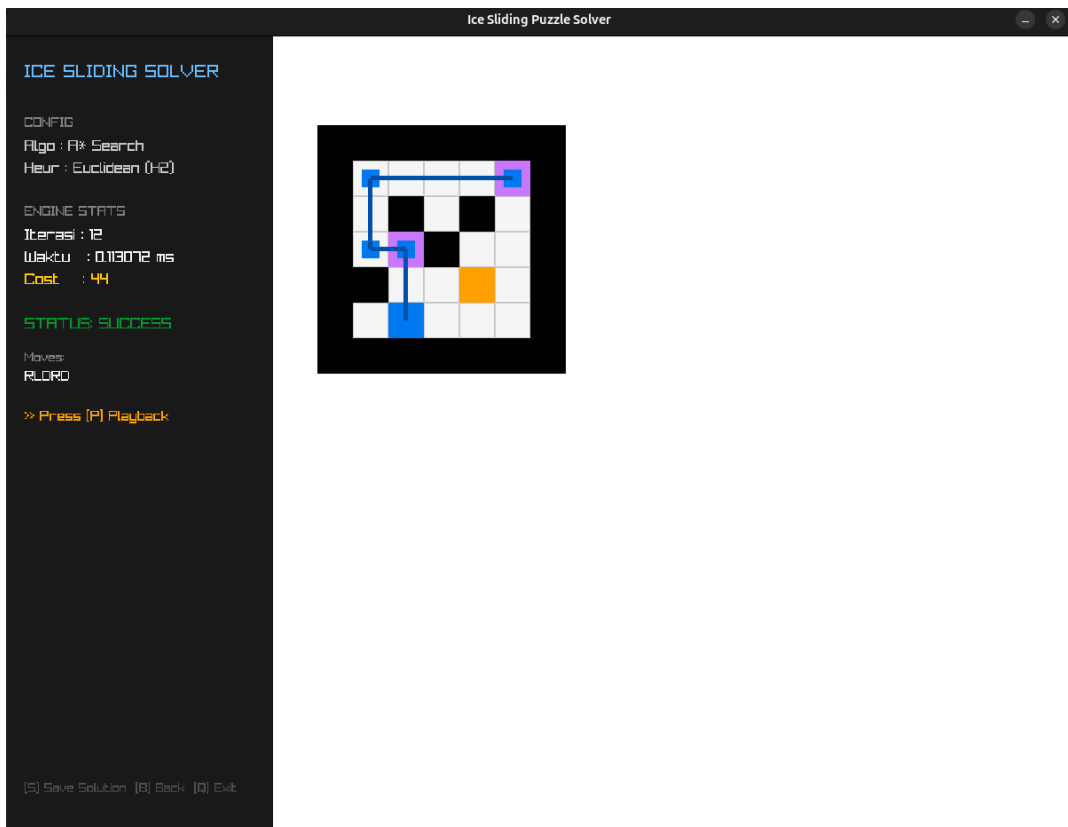
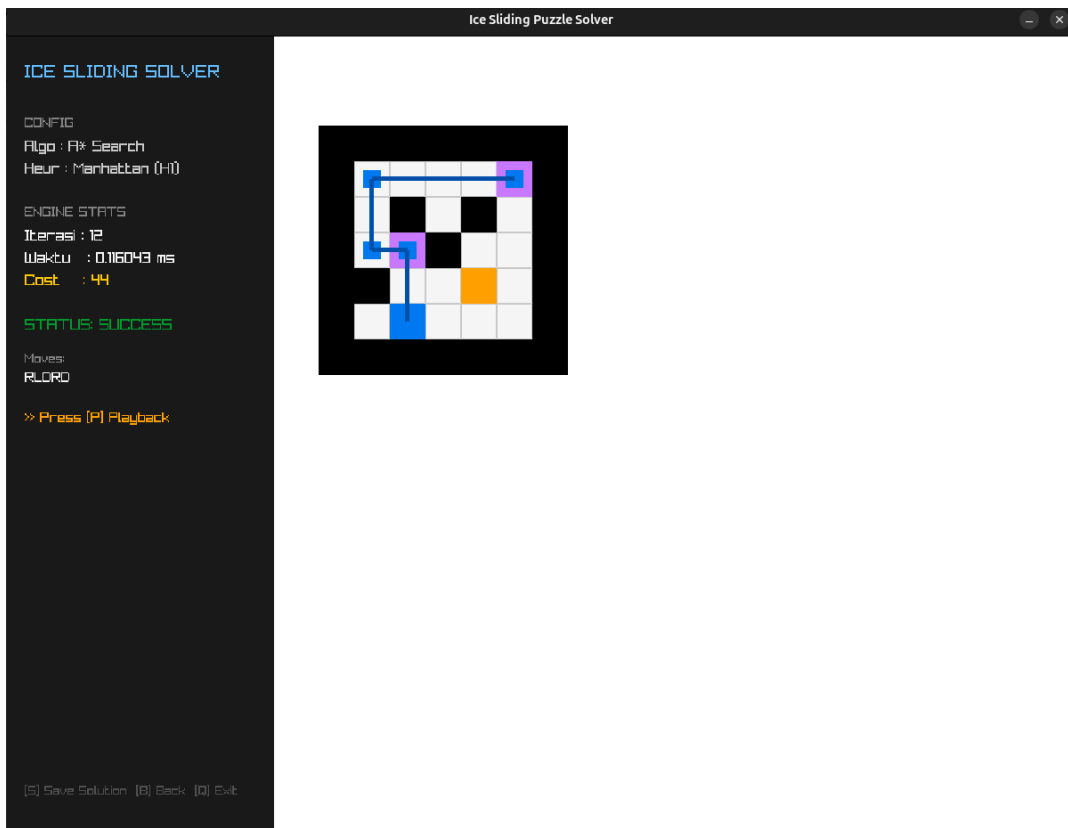
Input:

```

7 7
XXXXXXXXX
XZ...0X
X.X.X.X
X.1X..X
XX..L.X
X.O...X
XXXXXXXXX
999 999 999 999 999 999 999
999 1 5 2 8 1 999
999 7 999 4 999 6 999
999 2 1 999 5 4 999
999 6 1 7 999 2 999
999 9 1 3 4 8 999
999 999 999 999 999 999 999

```

Output:



ICE SLIDING SOLVER

CONFIG
Algo : A* Search
Heur : Chebyshev (H3)

ENGINE STATS
Iterasi : 12
Waktu : 0.120156 ms
Cost : 44

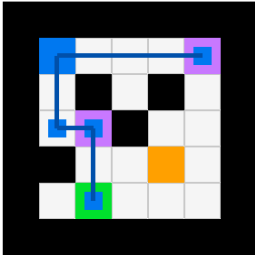
STATUS: SUCCESS

Moves
RLDRD

State: Initial
Speed: 2.0x
Status: PAUSED (SPACE)
Arrows: Step | </> Speed | ESC: Jump

(S) Save Solution (B) Back (Q) Exit

Ice Sliding Puzzle Solver



ICE SLIDING SOLVER

CONFIG
Algo : BGFS Search
Heur : Manhattan (H1)

ENGINE STATS
Iterasi : 6
Waktu : 0.065460 ms
Cost : 44

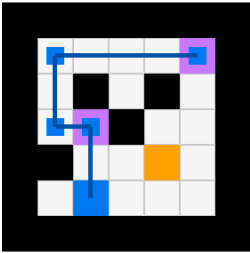
STATUS: SUCCESS

Moves
RLDRD

>> Press [P] Playback

(S) Save Solution (B) Back (Q) Exit

Ice Sliding Puzzle Solver



Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG
Algo : BFS Search
Heur : Euclidean (H2)

ENGINE STATS
Iterasi : 6
Waktu : 0.062953 ms
Cost : 44

STATUS: SUCCESS

Moves
RLDRD

>> Press [P] Playback

[S] Save Solution [B] Back [Q] Exit

Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG
Algo : BFS Search
Heur : Chebyshev (H3)

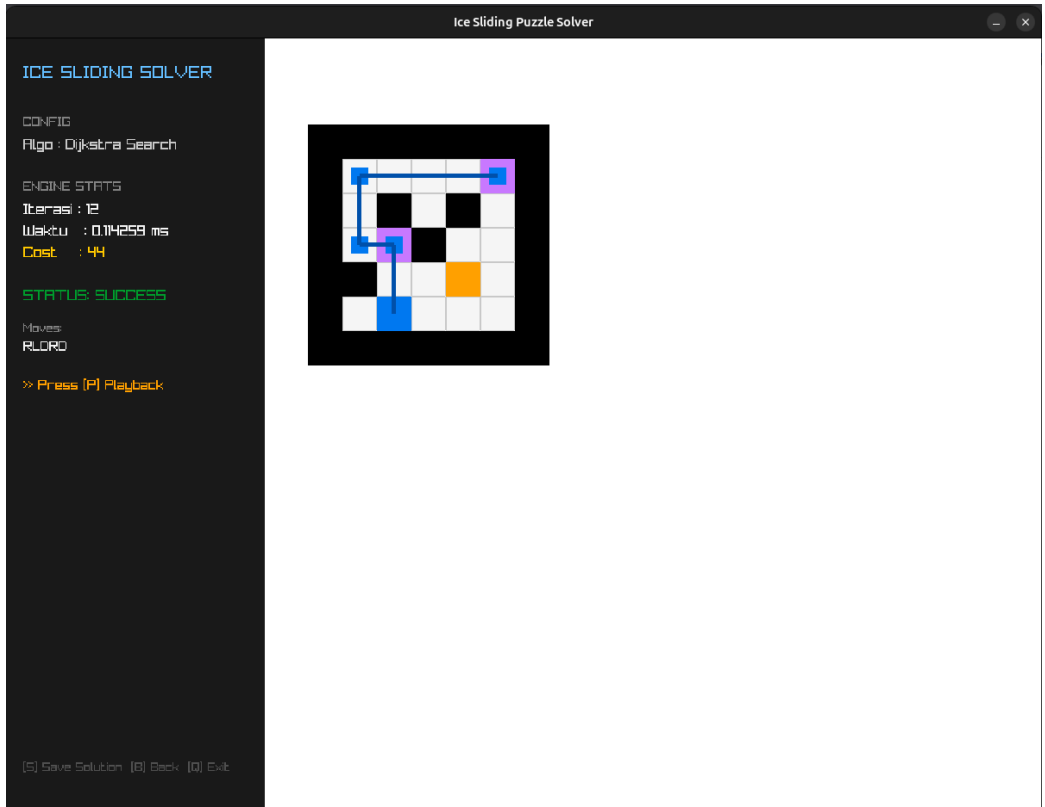
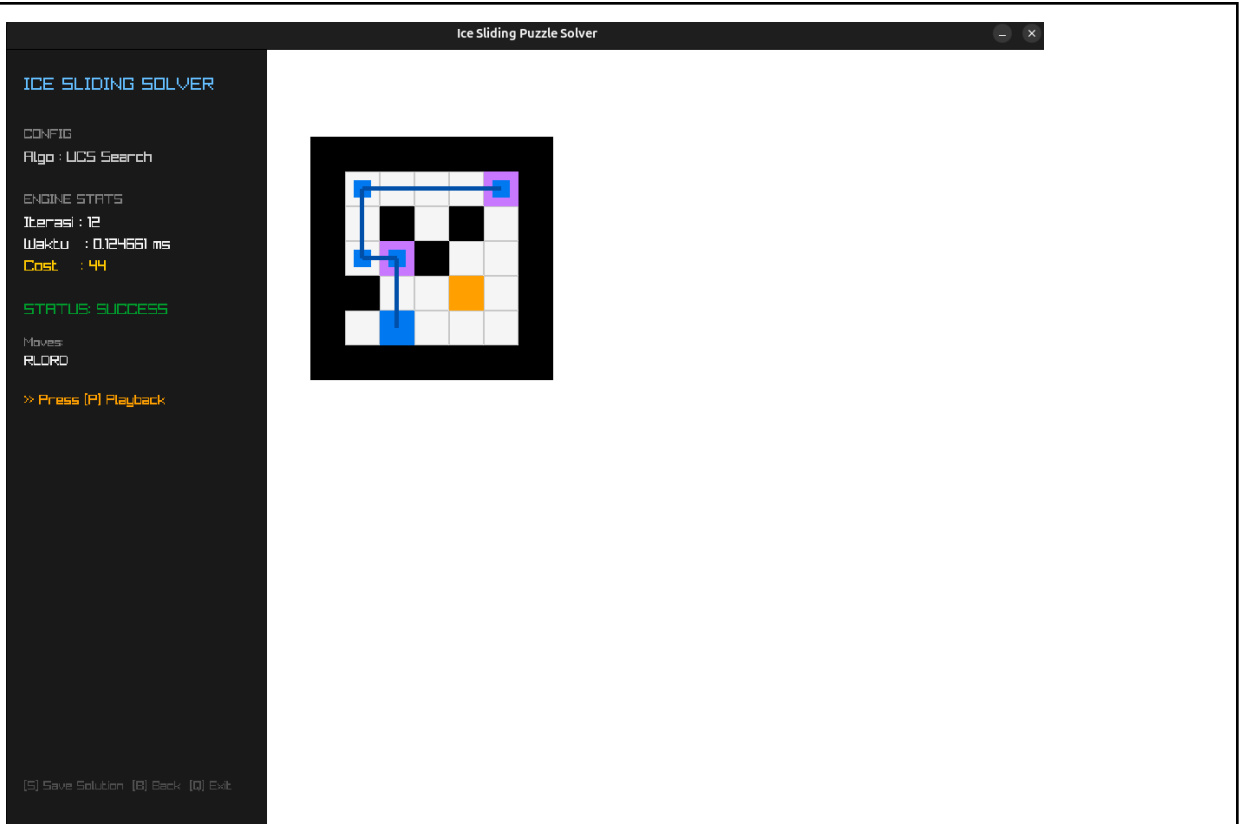
ENGINE STATS
Iterasi : 6
Waktu : 0.061058 ms
Cost : 44

STATUS: SUCCESS

Moves
RLDRD

>> Press [P] Playback

[S] Save Solution [B] Back [Q] Exit



Percobaan 4

Input:

12 12

XXXXXXXXXXXXX

XZLLX1*****X

X*XXL*XXLX*X

X*LLL*LLXX*X

X*XO*****X*X

X*LXL*XL*L*X

X*XXX*LL*X*X

X*LLL*LX*L*X

X*XXL***2X*X

X*XLLXXXLX*X

X*****0X

XXXXXXXXXXXXX

999 999 999 999 999 999 999 999 999 999 999

999 1 999 999 999 3 4 5 1 2 8 999

999 2 999 999 999 6 999 999 999 999 3 999

999 5 999 999 999 8 999 999 999 999 4 999

999 3 999 4 5 1 2 8 6 999 7 999

999 1 999 999 999 3 999 999 999 999 5 999

999 6 999 999 999 7 999 999 4 999 1 999

999 2 999 999 999 5 999 999 5 999 3 999

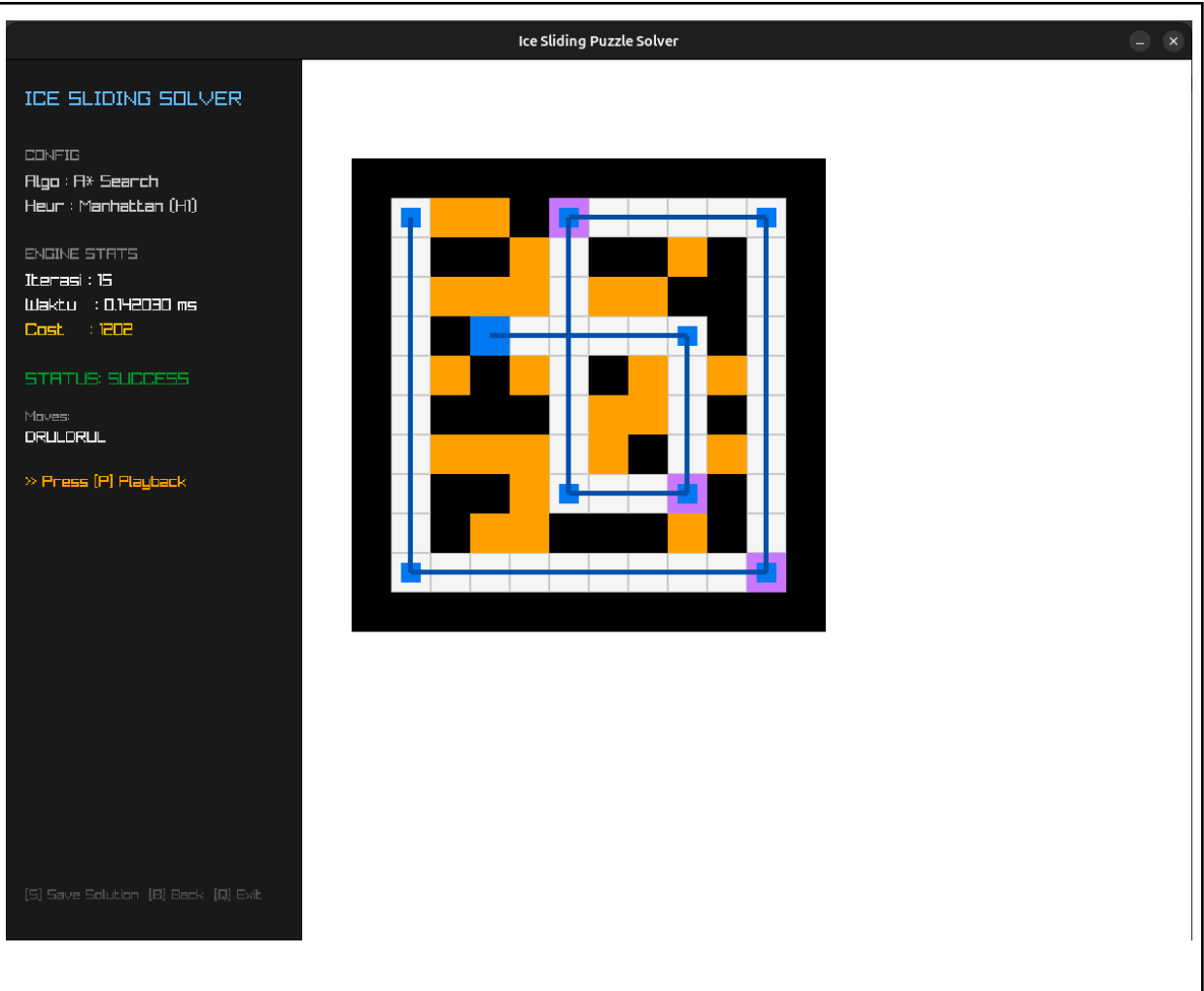
999 8 999 999 999 4 1 6 2 999 6 999

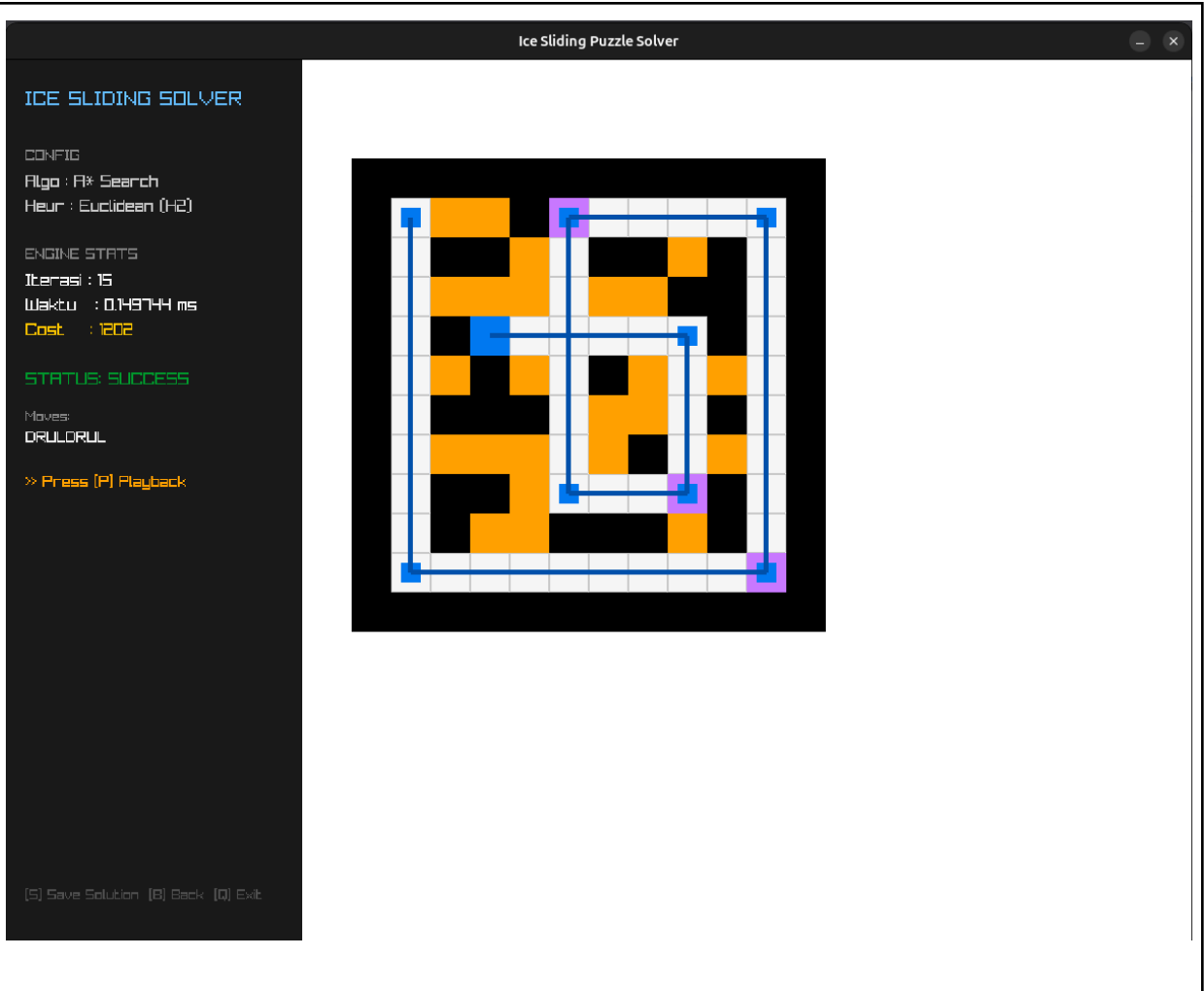
999 4 999 999 999 999 999 999 999 999 2 999

999 5 1 3 4 8 2 5 7 1 4 999

999 999 999 999 999 999 999 999 999 999 999

Output:





ICE SLIDING SOLVER

CONFIG

Algo : A* Search

Heur : Chebyshev (H3)

ENGINE STATS

Iterasi : 15

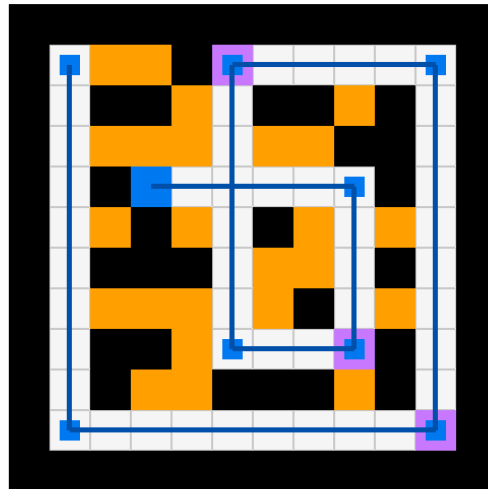
Waktu : 0.142212 ms

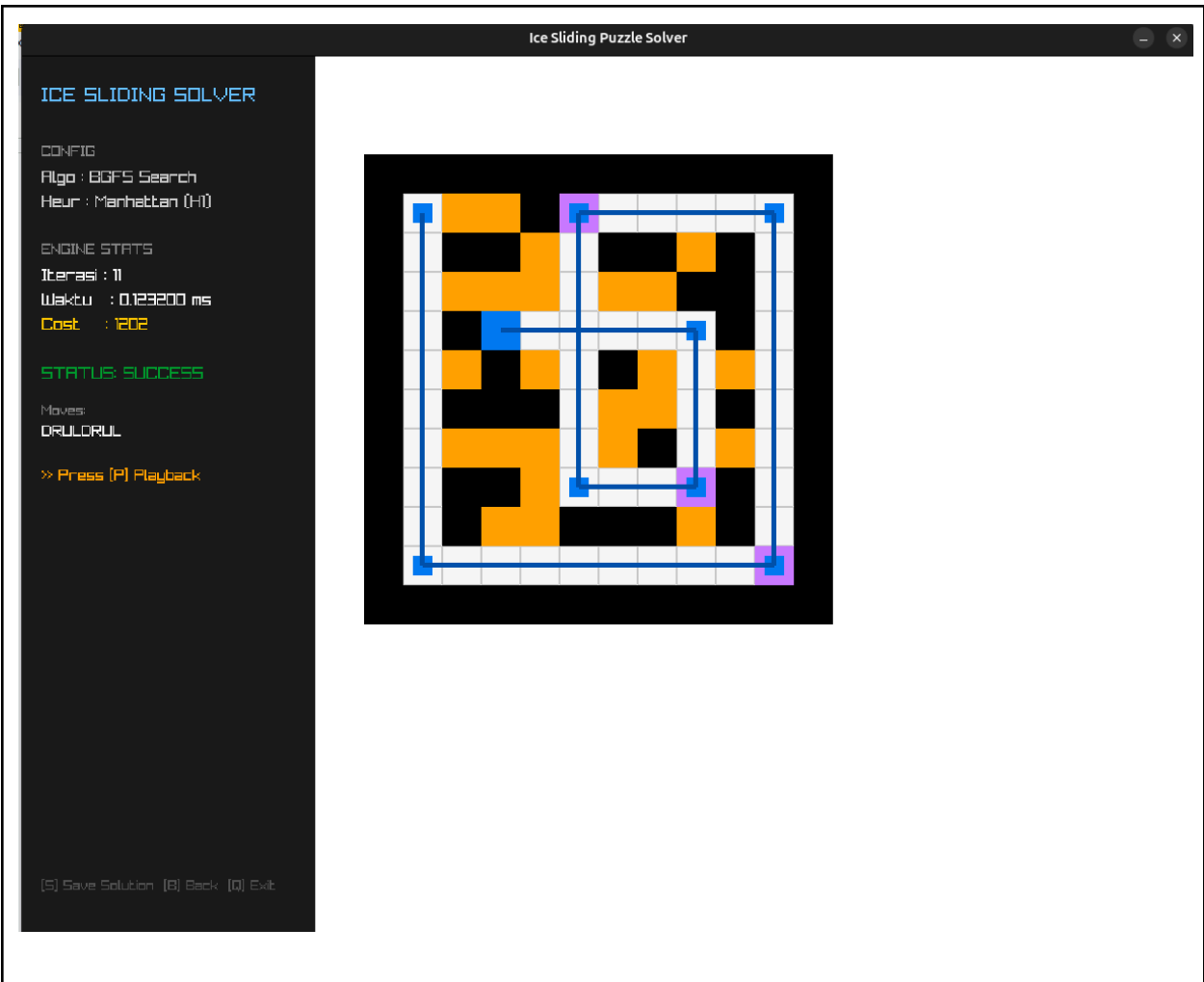
Cost : 1202

STATUS: SUCCESS

DRULDRUL

>> Press [P] Playback





Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG
Algo : BFS Search
Heur : Euclidean (H2)

ENGINE STATS
Iterasi : 9
Waktu : 0.117485 ms
Cost : 1202

STATUS: SUCCESS

Moves:
DRULORUL

>> Press [P] Playback

[S] Save Solution [B] Back [Q] Exit

Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG

Algo : BFS Search
Heur : Chebyshev (H3)

ENGINE STATS

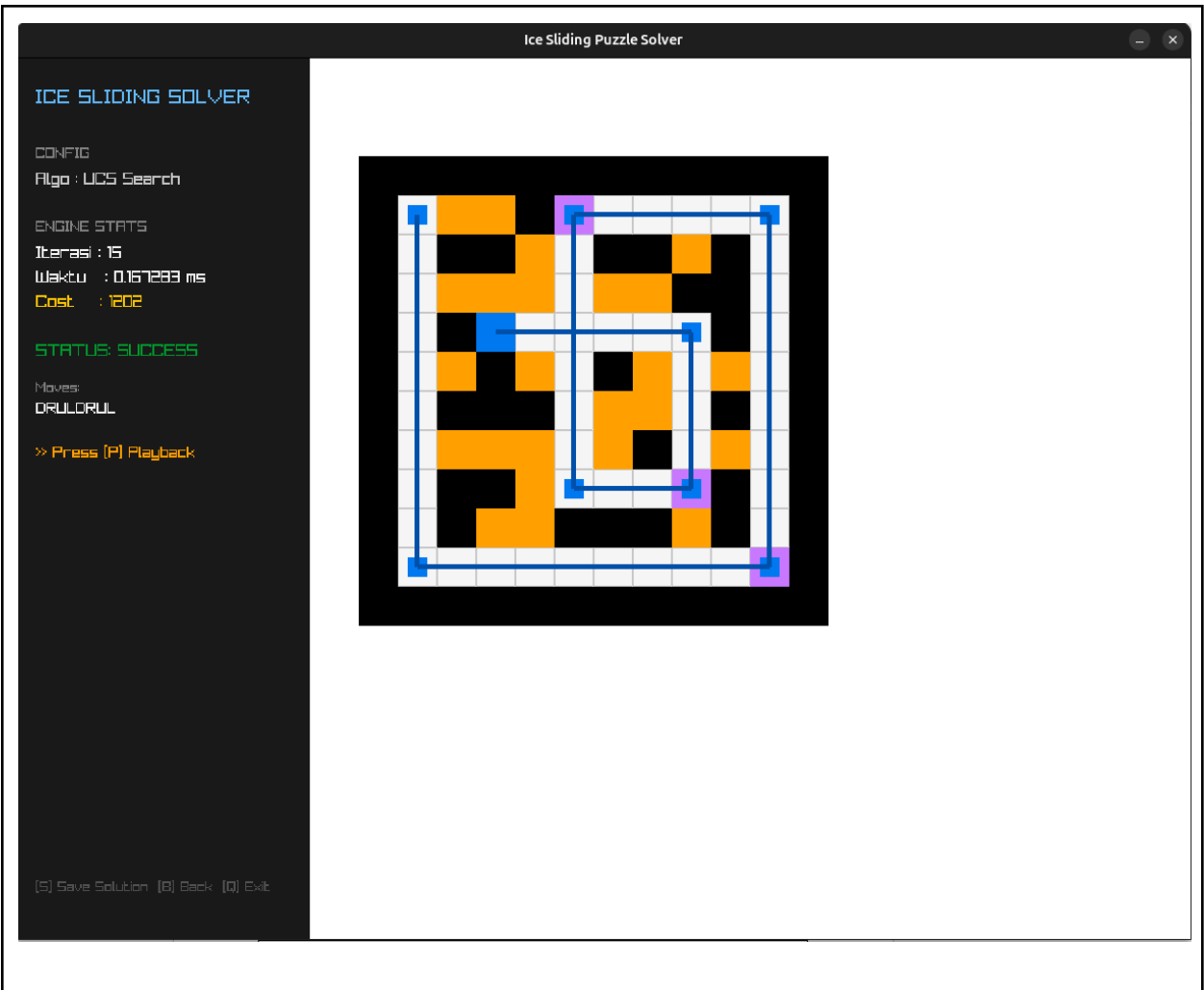
Iterasi : 9
Waktu : 0.130994 ms
Cost : 1202

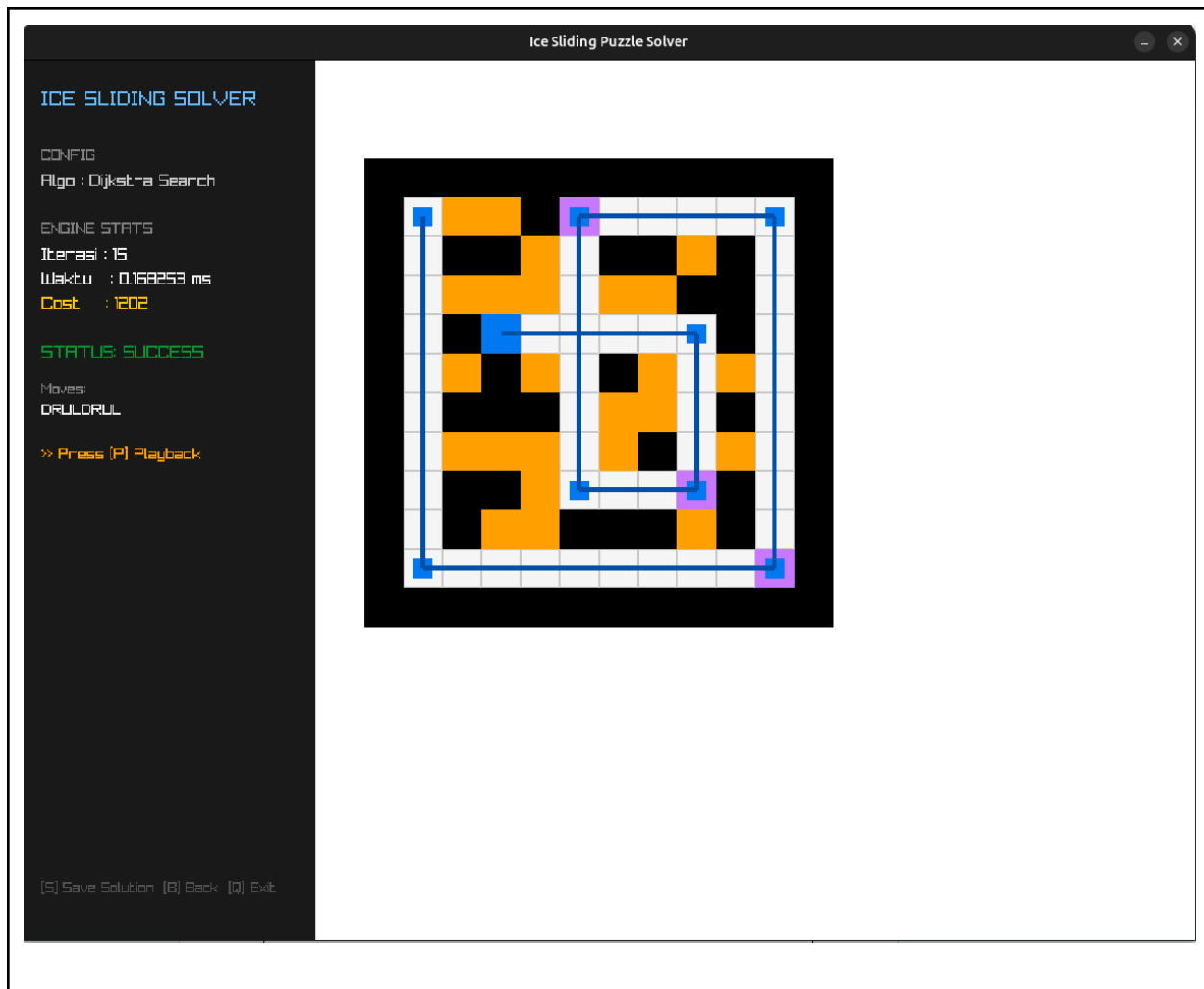
STATUS: SUCCESS

Moves:
DRULORUL

>> Press [P] Playback

[S] Save Solution [B] Back [Q] Exit





Percobaan 5

Input:

9 9

XXXXXXXXXX

XZ*XXXXX

X*XL***X

X*1***X

XL*XX*L

XL*L*OL

XL*L*X**

XX*****

XXXXXXXXXX

999 999 999 999 999 999 999 999

999 2 4 5 1 3 6 8 999

999 7 999 999 999 999 999 2 999

999 3 5 2 4 8 999 1 999

999 999 6 999 999 2 999 5 999

999 999 3 999 1 7 999 2 999
999 999 8 999 5 999 4 3 999
999 999 2 1 6 3 8 4 999
999 999 999 999 999 999 999 999

Output:

ICE SLIDING SOLVER

CONFIG

Algo : A* Search

Heur : Manhattan (H1)

ENGINE STATS

Iterasi : 11

Waktu : 0.096549 ms

Cost : 109

STATUS: SUCCESS

Moves:

ROLLUD

Step 6 : 0

Speed: 20x

Status: PAUSED (SPACE)

Arrows: Step | +/- Speed | ESC: Jump

[S] Save Solution [B] Back [Q] Exit

Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG
Algo : A* Search
Heur : Euclidean (H2)

ENGINE STATS
Iterasi : 12
Waktu : 0.11455 ms
Cost : 109

STATUS: SUCCESS

Moves:
ROLLUD

>> Press [P] Playback

(S) Save Solution (B) Back (Q) Exit

Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG
Algo : A* Search
Heur : Chebyshev (H3)

ENGINE STATS
Iterasi : 12
Waktu : 0.127504 ms
Cost : 109

STATUS: SUCCESS

Moves:
RDLURD

>> Press [P] Playback

(S) Save Solution (B) Back (Q) Exit

ICE SLIDING SOLVER

CONFIG

Algo : BGFS Search
Heur : Manhattan (H1)

ENGINE STATS

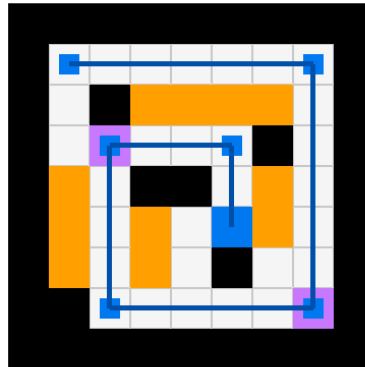
Iterasi : 7
Waktu : 0.071515 ms
Cost : 109

STATUS: SUCCESS

Moves:
RDLURD

>> Press [P] Playback

[S] Save Solution [B] Back [Q] Exit

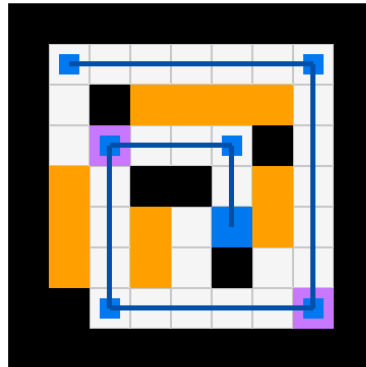


CONFIG

ENGINE STATS

STATUS: SUCCESS

>> Press [P] Playback



Ice Sliding Puzzle Solver

ICE SLIDING SOLVER

CONFIG
Algo : BGFS Search
Heur : Chebyshev (H3)

ENGINE STATS
Iterasi : 7
Waktu : 0.077838 ms
Cost : 109

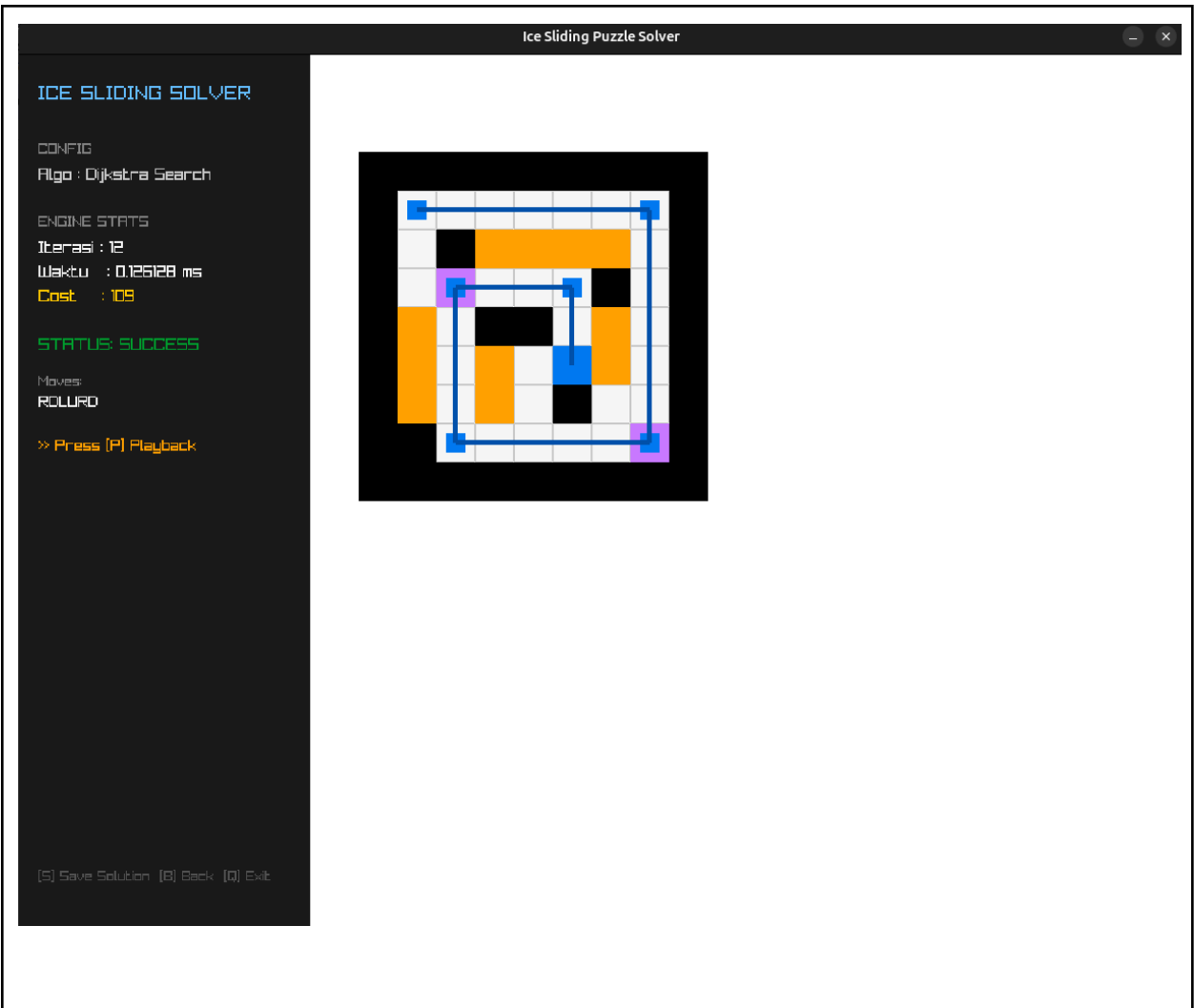
STATUS: SUCCESS

Moves:
RDLURD

Step 6 : 0
Speed: 20x
Status: PAUSED (SPACE)
Arrows: Step | +/- Speed | ESC: Jump

(S) Save Solution (B) Back (Q) Exit





Lampiran

Link Github : https://github.com/fariswirakusuma/Tucil3_13524130_13524144.git

Tabel Progress

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Faris Wirakusuma Triawan



Jonathan Harijadi